

Sistemas de Computación

Práctico # 4

Módulos de kernel y llamadas a sistema

Grupo: NoTengoGrupo

Alumnos:

- Bernardi Mateo
- Regnicoli Gustavo
- Schreiner Federico

Docentes:

- Miguel Angel Solinas
- Javier Alejandro Jorge

Año: 2026

Introducción:

¿Qué es exactamente un módulo del núcleo? Los módulos son fragmentos de código que se pueden cargar y descargar en el kernel según se requiera. Extienden la funcionalidad del kernel sin necesidad de reiniciar el sistema. Por ejemplo, un tipo de módulo es el controlador de dispositivo, que permite que el núcleo acceda al hardware conectado al sistema. Sin módulos, tendríamos que construir kernels monolíticos y agregar nuevas funciones directamente en la imagen del kernel. Además de tener kernels más grandes, esto tiene la desventaja de requerir que reconstruyamos y reiniciemos el kernel cada vez que queramos una nueva funcionalidad.

Desarrollo:

Pasos:

En primer lugar se instalan las dependencias y se clona el repositorio.

```
sudo apt-get install build-essential checkinstall linux-source
```

- **build-essential:** compilador GCC y herramientas
- **checkinstall:** crea paquetes .deb desde código fuente
- **linux-headers:** archivos necesarios para compilar módulos del kernel

```
git clone https://gitlab.com/sistemas-de-computacion-unc/kernel-modules.git
cd kernel-modules/part1/module
make
```

Este comando compila mimodulo.c y genera el archivo mimodulo.ko (la extensión .ko significa Kernel Object)

```
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ make
make -C /lib/modules/6.17.0-19-generic/build M=/home/gustavo/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module modules
make[1]: se entra en el directorio '/usr/src/linux-headers-6.17.0-19-generic'
make[2]: se entra en el directorio '/home/gustavo/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
You are using:          gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M]  mimodulo.o
mimodulo.c:8:5: warning: no previous prototype for 'modulo_lin_init' [-Wmissing-prototypes]
   8 | int modulo_lin_init(void)
     | ^~~~~~
mimodulo.c:17:6: warning: no previous prototype for 'modulo_lin_clean' [-Wmissing-prototypes]
  17 | void modulo_lin_clean(void)
     | ^~~~~~
MODPOST Module.symvers
CC [M]  mimodulo.mod.o
CC [M]  .module-common.o
LD [M]  mimodulo.ko
BTF [M] mimodulo.ko
Skipping BTF generation for mimodulo.ko due to unavailability of vmlinux
make[2]: se sale del directorio '/home/gustavo/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module'
make[1]: se sale del directorio '/usr/src/linux-headers-6.17.0-19-generic'
```

Se carga el módulo

```
sudo insmod mimodulo.ko
sudo dmesg | tail
```

```
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ sudo insmod mimodulo.ko
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ sudo dmesg | tail
[ 8835.831236] Local0: 0000000098554959 <Obj> Integer 0000000000000000

[ 8835.831246] No Arguments are initialized for method [APCL]

[ 8835.831249] ACPI Error: Aborting method \_SB.PCI0.LPC0.EC0.APCL due to previous error (AE_NOT_FOUND) (20250404/psparse-529)
[ 8835.831261] ACPI Error: Aborting method \_SB.PCI0.LPC0.EC0.PCLK due to previous error (AE_NOT_FOUND) (20250404/psparse-529)
[ 8835.831269] ACPI Error: Aborting method \_SB.PCI0.LPC0.EC0.QID due to previous error (AE_NOT_FOUND) (20250404/psparse-529)
[ 9027.252527] mimodulo: loading out-of-tree module taints kernel.
[ 9027.252537] mimodulo: module verification failed: signature and/or required key missing - tainting kernel
[ 9027.252784] Modulo cargado en el kernel.
```

En la captura de pantalla se observa la ejecución del comando `insmod`, el cual inyecta el módulo de forma dinámica directamente en el espacio de memoria del núcleo (Kernel Space). Mientras que `dmesg` accede de forma cronológica a los últimos registros del buffer de mensajes del sistema.

Se puede comprobar la correcta inicialización y ejecución de nuestro código al visualizarse el mensaje: “Módulo cargado en el kernel” al final del log.

Se verifica con `lsmod`

```
lsmod | head -10
```

```
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ lsmod | head -10
Module                  Size  Used by
mimodulo                12288  0
ums_realtek             24576  1
uas                     32768  0
usb_storage             86016  2 uas,ums_realtek
snd_seq_dummy           12288  0
ccm                     20480  3
rfcomm                  98304  16
snd_hrtimer             12288  1
qrtr                    53248  2
```

Se observa en la primera fila el módulo `mimodulo` con `Used by` en 0, lo que significa que está cargado pero ningún otro módulo depende de él.

El comando `modinfo` muestra los metadatos del módulo.

```
modinfo mimodulo.ko
```

Obteniendo:

```
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ modinfo mimodulo.ko
filename:                /home/gustavo/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module/mimodulo.ko
author:                  Catedra de SdC
description:              Primer modulo ejemplo
license:                  GPL
srcversion:               C6390D617B2101FB1B600A9
depends:
name:                    mimodulo
retpoline:                Y
vermagic:                 6.17.0-19-generic SMP preempt mod_unload modversions
```

Para finalizar el ciclo de vida del módulo dentro del sistema, se procede a su descarga utilizando el comando `rmmod`. Este comando le indica al núcleo que remueva el controlador de la memoria RAM de forma segura.

Se utilizó `dmesg` para ver el buffer del sistema cronológicamente.

```
sudo rmmod mimodulo
sudo dmesg | tail
```

```

gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ sudo rmmod mimodulo
[sudo] contraseña para gustavo:
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ sudo dmesg | tail
[30636.330629]
    Initialized Local Variables for Method [APCL]:
[30636.330631]   Local0: 00000000a94dcd22 <Obj>          Integer 0000000000000000

[30636.330651] No Arguments are initialized for method [APCL]

[30636.330655] ACPI Error: Aborting method \_SB.PCI0.LPC0.EC0.APCL due to previous error (AE_NOT_FOUND) (20250404/psparse-529)
[30636.330677] ACPI Error: Aborting method \_SB.PCI0.LPC0.EC0.PCLK due to previous error (AE_NOT_FOUND) (20250404/psparse-529)
[30636.330695] ACPI Error: Aborting method \_SB.PCI0.LPC0.EC0.QID due to previous error (AE_NOT_FOUND) (20250404/psparse-529)
[30964.930648] Modulo descargado del kernel.

```

Ahora se modifica el archivo `mimodulo.c` para que use la etiqueta del nombre de grupo. Se usa `printk`, que escribe directamente en el log del kernel, no se puede usar `printf` porque esa función pertenece a la biblioteca estándar de C (`libc`), la cual solo está disponible en el espacio de usuario.

```
nano mimodulo.c
```

```

GNU nano 7.2                                                                    mimodulo.c *
#include <linux/module.h>                /* Requerido por todos los módulos */
#include <linux/kernel.h>                /* Definición de KERN_INFO */
MODULE_LICENSE("GPL"); /* Licencia del modulo */
MODULE_DESCRIPTION("Primer modulo ejemplo");
MODULE_AUTHOR("Catedra de Sdec");

/* Función que se invoca cuando se carga el módulo en el kernel */
int modulo_lin_init(void)
{
    printk(KERN_INFO "Modulo cargado en el kernel por NoTengoGrupo.\n");

    /* Devolver 0 para indicar una carga correcta del módulo */
    return 0;
}

/* Función que se invoca cuando se descarga el módulo del kernel */
void modulo_lin_clean(void)
{
    printk(KERN_INFO "Modulo descargado del kernel por NoTengoGrupo.\n");
}

/* Declaración de funciones init y exit */
module_init(modulo_lin_init);
module_exit(modulo_lin_clean);

```

Se realiza los mismo paso y se comprueba:

```

gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ sudo rmmod mimodulo
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ sudo insmod mimodulo.ko
gustavo@gustavo-Lenovo-V145-15AST:~/noTengoGrupo-SdC/tp4_modelos_kernel/kernel-modules/part1/module$ sudo dmesg | grep -i NoTengoGrupo
[33060.542346] Modulo cargado en el kernel por NoTengoGrupo.
[33087.319402] Modulo descargado del kernel por NoTengoGrupo.

```

Desafío #1

¿Qué es checkinstall?

Checkinstall es una utilidad de línea de comandos para sistemas operativos tipo Unix/Linux diseñada para interceptar el proceso de instalación tradicional de un software compilado desde el código fuente (que usualmente concluye con el comando `make install`).

En lugar de copiar directamente los archivos binarios, librerías y páginas de manual en los directorios del sistema (como `/usr/local/bin` o `/usr/local/lib`) sin control centralizado, checkinstall realiza un seguimiento de cada archivo creado o modificado durante la instalación. Al finalizar, genera automáticamente un paquete de software nativo para el gestor de paquetes de la distribución en uso (un archivo `.deb` para distribuciones basadas en Debian/Ubuntu, un `.rpm` para Red Hat/Fedora/SUSE, o un `.tgz` para Slackware) y luego lo instala utilizando el gestor del sistema (`dpkg`, `rpm`, etc.).

¿Para qué sirve?

- **Mantenimiento y limpieza del sistema:** Evita que queden archivos "huérfanos" dispersos por el sistema de archivos que luego sean difíciles de rastrear y eliminar de forma manual.
- **Desinstalación y actualización sencilla:** Al estar registrado en la base de datos del gestor de paquetes nativo, el software instalado desde el código fuente se puede remover por completo con un simple comando (ej. `sudo apt remove <nombre>`) o actualizar limpiamente.
- **Portabilidad:** Permite empaquetar una aplicación compilada en una máquina y distribuirla fácilmente en otros equipos de la misma arquitectura sin necesidad de volver a compilar.

Empaquetado de un "Hello World" con checkinstall

El programa y su MakeFile se encuentran dentro del repositorio adjunto.

```
mateo@mateo-HP-Laptop-15-da0xxx:~$ helloworld
Hola somos NoTengoGrupo!
mateo@mateo-HP-Laptop-15-da0xxx:~$ dpkg -l | grep -i hello
ii helloworld-custom                                1.0.0-1
                                amd64             Package created with checkinstall 1.6.3
mateo@mateo-HP-Laptop-15-da0xxx:~$
```

Acciones para mejorar la seguridad del kernel: Restricción de módulos no firmados y rootkits

Los *Loadable Kernel Modules* (LKM) maliciosos son una de las amenazas más críticas en entornos Linux. Un atacante con privilegios de superusuario (`root`) que logre insertar un módulo malicioso en el espacio del kernel obtiene control total sobre el hardware y el sistema operativo (Ring 0). Un rootkit de este tipo puede secuestrar la tabla de llamadas al sistema (*Syscall Table*), alterando funciones clave para ocultar su propia existencia, ocultar procesos maliciosos, interceptar tráfico de

red o camuflar archivos maliciosos en el disco, haciendo que herramientas de auditoría estándar como **ps**, **ls** o **netstat** sean completamente engañadas.

Acciones bibliográficas y de configuración para mitigar el riesgo

Para neutralizar la inserción de LKM maliciosos, la acción de seguridad primordial consiste en exigir la validación criptográfica estricta de firmas en todos los módulos.

1. Activación de firmas obligatorias en la compilación del Kernel (CONFIG_MODULE_SIG_FORCE):

Al configurar un kernel de Linux con el parámetro `CONFIG_MODULE_SIG_FORCE=y`, el núcleo implementa una política restrictiva donde se niega categóricamente la carga de cualquier módulo que no cuente con una firma digital válida y verificable. Si un módulo es modificado por un tercero o carece de firma, el comando `insmod` o `modprobe` fallará inmediatamente con un error de ejecución (Required key not available), impidiendo que incluso el usuario root inyecte código no verificado.

2. Implementación de Secure Boot (Arranque Seguro):

Mecanismo de hardware/firmware que asegura que el cargador de arranque (*bootloader*) y el kernel cargado estén firmados por autoridades de confianza. Cuando Secure Boot está activo en la UEFI, el kernel de Linux entra automáticamente en un estado de **Restricción (Lockdown Mode)**, bloqueando características que permiten modificar el código del kernel en caliente (como el acceso directo a `/dev/mem` o la inyección de módulos no firmados).

3. Uso de llaves MOK (Machine Owner Key):

Permite al administrador del sistema generar su propio par de claves criptográficas asimétricas, inscribir la clave pública en la base de datos de la UEFI mediante la herramienta `mokutil`, y firmar legítimamente los módulos propios o de terceros necesarios para la operación (por ejemplo, drivers de VirtualBox o NVIDIA).

Desafío #2

¿Qué funciones tiene disponible un programa y un módulo?

Espacio de Usuario (User Space) vs. Espacio del Kernel (Kernel Space)

- **Programa (Espacio de Usuario):** Se ejecuta en el modo protegido y restringido del procesador (**Ring 3**). No tiene acceso directo a los recursos de hardware ni a la memoria física global. Para interactuar con el exterior, depende exclusivamente de la API de la librería estándar de C (glibc) y de las **llamadas al sistema (System Calls)**, las cuales actúan como un puente seguro y controlado hacia el kernel. Su flujo de ejecución está delimitado bajo un punto de entrada principal determinado por la función `main()`.
- **Módulo (Espacio del Kernel):** Se ejecuta con los máximos privilegios disponibles (**Ring 0**). Carece de acceso a las librerías estándar de espacio de usuario (no puede usar `printf`, `malloc`, etc.), pero tiene a su disposición todas las funciones internas y símbolos exportados por el propio núcleo de Linux (como `printk` para bitácoras, `kmalloc` para asignación de memoria del núcleo, o `copy_to_user/copy_from_user` para el intercambio seguro de flujos de datos entre espacios de memoria). No tiene un bucle principal de ejecución; en su

lugar, expone funciones de eventos: una rutina de inicialización (module_init) y otra de salida/limpieza (module_exit).

Espacio de Datos

- En el **Espacio de Usuario**, cada proceso vive dentro de su propio mapa de memoria virtual privado provisto y aislado por la Unidad de Gestión de Memoria (MMU). Dos procesos independientes pueden apuntar conceptualmente a la misma dirección virtual interna pero accederán a celdas físicas totalmente aisladas.
- En el **Espacio del Kernel**, todos los módulos activos comparten un único espacio de direccionamiento virtual global. Un error de asignación o una escritura corrupta en la memoria por parte de un módulo repercute directamente sobre toda la estructura de datos del sistema operativo, con consecuencias globales.

Drivers e Investigación del Contenido de /dev

El directorio /dev no se aloja de manera física persistente en el disco; es un sistema de archivos virtual montado en memoria conocido como devtmpfs. Contiene archivos especiales denominados **nodos de dispositivos (device nodes)** que sirven como interfaces lógicas estandarizadas para comunicarse con los drivers residentes en el kernel de Linux, implementando la premisa de Unix de que *"todo es un archivo"*.

Al realizar un listado detallado (ls -l /dev), se distinguen principalmente dos categorías a través del primer carácter de sus permisos de acceso:

1. **Dispositivos de Caracteres (c):** Transmiten flujos de información secuencial de manera asíncrona, byte por byte, sin almacenamiento intermedio (ej. puertos serie, terminales /dev/ttyX, fuentes de datos aleatorios /dev/urandom, o el sumidero de datos /dev/null).
2. **Dispositivos de Bloques (b):** Gestionan el direccionamiento de datos organizados en sectores o bloques de tamaño fijo (por lo general, de 512 o 4096 bytes), permitiendo accesos aleatorios de lectura/escritura (ej. unidades de almacenamiento masivo de estado sólido o mecánicos como /dev/sda o /dev/nvme0n1).

Adicionalmente, cada nodo en /dev posee dos identificadores numéricos específicos:

- **Número Mayor (Major Number):** Indica explícitamente al subsistema del kernel qué driver específico debe hacerse cargo de procesar las operaciones de Entrada/Salida sobre ese nodo.
- **Número Menor (Minor Number):** Identifica a la subunidad o instancia de hardware específica que debe ser atendida por dicho driver común.

1. ¿Qué diferencias se pueden observar entre los dos modinfo ?

Al ejecutar la herramienta de inspección modinfo sobre dos módulos diferentes (por ejemplo, uno distribuido oficialmente por el equipo de seguridad de la distribución y otro compilado a mano localmente sin aplicar procesos adicionales), se constatan variaciones estructurales en sus metadatos:

Campo de Metadata	Módulo Oficial / Firmado	Módulo Personalizado / Sin Firmar
filename	Ruta absoluta en /lib/modules/...	Ruta local del archivo compilado .ko
vermagic	Coincide estrictamente con la versión del kernel oficial	Puede diferir si las cabeceras sufrieron modificaciones
signer	Presente (ej. Ubuntu SMP Signed Kernel Key)	Ausente por completo
sig_key	Presente (Huella digital criptográfica de la clave)	Ausente por completo
sig_hashalgo	Presente (Indica algoritmo de hashing, ej. sha256)	Ausente por completo
signature	Bloque de bytes con la firma criptográfica válida	Ausente por completo

2. ¿Qué divers/modulos estan cargados en sus propias pc? (Análisis de lsmod y diff)

El comando lsmod lee la información en tiempo real provista por el archivo virtual /proc/modules para listar los módulos cargados en memoria.

Explicación de las diferencias de hardware habituales

Las discrepancias que surgen al realizar una comparación cruzada entre distintas estaciones de trabajo corresponden a las diferencias de los componentes electrónicos de los equipos. Por ejemplo:

- **Subsistema de Gráficos:** Un equipo con GPU dedicada cargará módulos privativos o libres de video como nvidia, nvidia_modeset o nouveau, mientras que un equipo con gráficos integrados Intel o AMD cargará drivers específicos de arquitectura como i915 o amdgpu.
- **Procesadores:** Dependiendo del fabricante de la CPU, se instanciarán extensiones de virtualización por hardware diferentes: kvm_intel junto con vboxdrv para arquitecturas Intel, frente a kvm_amd para plataformas basadas en AMD.
- **Interfaces de Red:** Adaptadores cableados o inalámbricos disímiles requerirán módulos correspondientes como r8169 (Realtek) o iwlwifi (Intel Wireless).

3. ¿Cuales no están cargados pero están disponibles?

Los módulos instalados que no se encuentran en ejecución activa en la memoria RAM están guardados en el almacenamiento secundario, organizados bajo la ruta estructurada: `/lib/modules/$(uname -r)/kernel/`

El subsistema encargado de la detección automática (como udev junto con modprobe) solo invoca e introduce los módulos cuando el bus de datos (PCI, USB) notifica la conexión física del hardware compatible (emparejamiento por alias de hardware).

¿Qué sucede cuando el driver de un dispositivo no está disponible?

Si se conecta un periférico o componente interno y el kernel carece de un módulo compatible (ya sea porque no se incluyó en la compilación del núcleo o porque faltan paquetes de firmware específicos):

1. El hardware permanecerá en estado inactivo o inoperante. Las capas superiores de software no podrán interactuar con él.
2. No se creará ningún nodo de acceso de archivos representativo dentro del directorio `/dev`.
3. Herramientas de diagnóstico de bajo nivel como `lspci -v` o `lsusb` detectarán la presencia física del chip en el bus de datos, pero en el campo asignado a los drivers mostrarán líneas vacías o advertencias genéricas como: `Kernel driver in use: UNKNOWN` o simplemente omitirán la asignación del controlador.

4. Correr hwinfo en una pc real con hw real y agregar la url de la información de hw en el reporte.

hwinfo es una herramienta avanzada de diagnóstico para sistemas Linux que realiza un escaneo exhaustivo de los componentes de hardware del sistema. A diferencia de comandos básicos, interroga directamente a los buses de comunicación para extraer detalles técnicos exactos de los componentes.

Para exportar un reporte compacto y legible sin saturar la salida de logs, se ejecuta el comando redirigiendo el resultado a un archivo de texto:

```
hwinfo --short > reporte_hardware.txt
```

Este archivo almacena la topología estructurada del equipo (CPU, memoria, almacenamiento, controladores de video, tarjetas de red y sonido). Posteriormente, para su inclusión en repositorios y reportes compartidos de auditoría, se sube el contenido a un servicio de almacenamiento compartido o Pastebin corporativo para referenciar mediante su correspondiente URL de acceso.

5. ¿Qué diferencia fundamental existe entre un módulo y un programa?

A partir de los fundamentos de programación del kernel de Linux, las diferencias estructurales clave se resumen en la siguiente tabla:

Criterio de Comparación	Programa Convencional	Módulo del Kernel (LKM)
Espacio de Ejecución	Espacio de Usuario (User Space - Ring 3).	Espacio del Kernel (Kernel Space - Ring 0).
Punto de Entrada	Función <code>main()</code> . Se autoejecuta secuencialmente.	Función declarada mediante <code>module_init()</code> .
Finalización	Termina al completar sus instrucciones o llamar a <code>exit()</code> .	Permanece residente en memoria atendiendo llamadas hasta que se remueva explícitamente (<code>module_exit</code>).
Vinculación de Librerías	Enlace dinámico o estático con librerías estándar como <code>libc</code> .	Enlace exclusivo contra la tabla interna de símbolos expuestos del núcleo (<code>/proc/kallsyms</code>).
Gravedad ante un Fallo	Provoca el cierre del proceso (<i>Crash / Segmentation Fault</i>). El sistema general se mantiene estable.	Provoca una inestabilidad severa conocida como <i>Kernel Oops</i> o un congelamiento total del sistema operativo (<i>Kernel Panic</i>).

6. ¿Cómo puede ver una lista de las llamadas al sistema que realiza un simple helloworld en c?

Para visualizar en tiempo real la lista completa de interacciones (llamadas al sistema) que solicita un binario ejecutable elemental hacia el núcleo de Linux, se implementa el comando `strace`.

Dado un archivo ejecutable compilado llamado `helloworld`, se ejecuta de la siguiente manera:

Flujo básico capturado por el comando

Al iniciar el rastreo, se podrá observar una secuencia lógica de llamadas de bajo nivel:

1. `execve()`: Invoca la carga y ejecución del binario en un nuevo proceso.
2. `brk()` / `mmap()`: Solicitudes iniciales al kernel para reservar y mapear los bloques de memoria virtual requeridos para el espacio de datos y carga de librerías dinámicas (`libc.so`).
3. `openat()` / `read()`: Apertura y lectura de archivos de configuración y librerías del sistema.
4. `write()`: La llamada crítica encargada de materializar el mensaje. Recibe como descriptor de archivo el número 1 (correspondiente a la salida estándar `stdout`), el puntero hacia la cadena de caracteres del saludo y la longitud de la palabra.

7. ¿Qué es un segmentation fault?

Un Segmentation Fault (Fallo de Segmentación / Violación de Acceso) se genera cuando un programa que se ejecuta en el espacio de usuario intenta realizar una operación no permitida en la memoria física a través de su dirección virtual. Esto ocurre comúnmente al desreferenciar un puntero que apunta a `NULL`, intentar escribir datos en segmentos declarados como de "Solo Lectura" (como el segmento de código o texto), o al exceder los límites asignados en el direccionamiento de buffers.

Manejo por parte del Hardware y el Kernel

1. Cuando la CPU procesa la instrucción de acceso inválida, la Unidad de Gestión de Memoria (**MMU**) detecta que no existe una traducción de página válida o que se violaron los bits de privilegios de la memoria.
2. La MMU detiene inmediatamente la ejecución e interrumpe al procesador lanzando una excepción de hardware (*Page Fault* o Fallo de Página inválido).
3. El **Kernel** toma el control del hilo de ejecución a través de su manejador de interrupciones. Identifica el proceso causante del fallo y le envía de manera asíncrona la señal del sistema **SIGSEGV** (Signal número 11).
4. Por defecto, si el programa de usuario no tiene una rutina especializada para procesar dicha señal, el kernel da de baja el proceso de forma inmediata y escribe un reporte en los logs del sistema (`dmesg`), pudiendo opcionalmente generar un volcado completo del estado de la memoria en disco (*Core Dump*) para propósitos de depuración.

Manejo por parte de un Programa de Usuario

Un desarrollador puede interceptar esta señal implementando las funciones de la librería estándar `signal()` o de manera más robusta `sigaction()`. Al capturar `SIGSEGV`, el programa ejecuta un método diseñado a medida que puede registrar el error en un archivo de logs propio o realizar rutinas de cierre controlado de bases de datos. No obstante, recuperar el flujo normal y continuar la ejecución del programa después de un fallo de segmentación no es recomendable, dado que el estado interno de la memoria ha quedado corrompido.

8. Guía técnica paso a paso para la compilación, firma y despliegue de un módulo personalizado

A continuación se detalla el proceso completo para compilar un módulo de kernel propio que registre el nombre del equipo, seguido de los pasos criptográficos para firmarlo bajo entornos con Secure Boot activo utilizando el script `sign-file` y la utilidad de llaves de propietario de máquina (MOK).

Paso 1: Desarrollo del Módulo en C (`mimodulo.c`) que está cargado en el repositorio adjuntado en la entrega.

Paso 2: Makefile para Compilación también en el repositorio anterior.

Paso 3: Generación del par de claves criptográficas de confianza (MOK)

```
~$ openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der -nodes -days 36500 -subj "/CN=VMware/"
```

Paso 4: Firma criptográfica del archivo `.ko`

```
~$ sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ./MOK.priv ./MOK.der $ (modinfo -n mimodulo.ko)
```

Paso 5: Importación de la Clave Pública al Llavero del Firmware UEFI

```
~$ sudo mokutil --import MOK.der
```

Paso 6: Verificación del estado de la firma del módulo

```
mateo@mateo-HP-Laptop-15-da0xxx:~$ cd noTengoGrupo-SdC/tp4_modulos/MiPrimerModulo
mateo@mateo-HP-Laptop-15-da0xxx:~/noTengoGrupo-SdC/tp4_modulos/MiPrimerModulo$ modinfo mimodulo.ko
filename:           /home/mateo/noTengoGrupo-SdC/tp4_modulos/MiPrimerModulo/mimodulo.ko
description:        Un módulo de ejemplo para Linux
author:             NoTengoGrupo
license:            GPL
srcversion:         9A20CE01BASE64499F77DDA
depends:
name:               mimodulo
retpoline:          Y
vermagic:           6.17.0-29-generic SMP preempt mod_unload modversions
stg_id:             PKCS#7
signer:             VMware
stg_key:            79:25:ED:F5:B3:E7:BF:E6:6B:84:22:72:A5:19:6A:67:03:B4:EF:F7
stg_hashalgo:       sha256
signature:          1F:C0:F2:85:4E:60:82:27:74:68:1E:C6:CC:34:5A:23:72:A9:25:39:
9A:65:96:2B:AE:C8:86:C2:5B:7F:AA:7D:FA:F8:44:48:32:86:CF:D5:
39:CC:B3:CF:B0:21:3F:FF:85:5C:0D:02:69:64:BC:B8:A6:30:16:6E:
C0:8D:21:01:E5:49:5B:4B:AD:D3:15:67:24:26:6E:43:37:6C:59:59:
5E:B8:36:B4:0C:2C:7B:1F:93:86:FA:B5:26:10:A7:55:F8:A7:E2:00:
3E:5C:0E:2A:8B:A2:53:56:59:08:47:01:D3:C8:D0:D1:4D:02:C3:02:
E0:A5:CF:BB:A2:00:F8:8D:54:DC:55:C3:21:72:7D:D3:AA:5E:2C:58:
CA:B2:21:4F:9F:EC:ED:09:A5:29:2E:72:6A:ED:B0:85:A6:2B:A2:C8:
26:B6:96:3F:7C:D3:A7:80:48:21:37:05:16:C4:A4:70:79:1A:44:0B:
AD:A5:D2:5C:03:59:EA:B2:23:B5:B6:5A:92:F8:65:A7:68:6D:26:98:
66:D6:DE:2E:D5:04:B8:38:01:2B:91:03:FE:F6:FE:69:41:E8:10:8B:
1F:19:8E:A5:A9:AE:AB:8E:53:E4:E4:70:E3:39:49:2C:15:AD:A2:33:
98:8B:2F:2E:C2:29:73:38:3A:EE:B1:94:49:18:66:64
mateo@mateo-HP-Laptop-15-da0xxx:~/noTengoGrupo-SdC/tp4_modulos/MiPrimerModulo$
```

Paso 7: Carga, descarga y validación de registros (Evidencia de ejecución)

```
mateo@mateo-HP-Laptop-15-da0xxx:~/noTengoGrupo-SdC/tp4_modulos/MiPrimerModulo$ sudo rmmod mimodulo
[sudo] password for mateo:
rmmod: ERROR: Module mimodulo is not currently loaded
mateo@mateo-HP-Laptop-15-da0xxx:~/noTengoGrupo-SdC/tp4_modulos/MiPrimerModulo$ sudo insmod mimodulo.ko
mateo@mateo-HP-Laptop-15-da0xxx:~/noTengoGrupo-SdC/tp4_modulos/MiPrimerModulo$ lsmod | grep mimod
mimodulo                12288      0
mateo@mateo-HP-Laptop-15-da0xxx:~/noTengoGrupo-SdC/tp4_modulos/MiPrimerModulo$ sudo rmmod mimodulo
```

10. ¿Qué pasa si mi compañero con Secure Boot habilitado intenta cargar un módulo firmado por mí?

Respuesta y Justificación: Si un compañero descarga el binario modulo_equipo.ko e intenta cargarlo directamente en su sistema operativo ejecutando `sudo insmod`, el kernel **rechazará la operación inmediatamente**, impidiendo la carga del módulo y arrojando un error crítico en la consola similar a: `insmod: ERROR: could not insert module modulo_equipo.ko: Required key not available o Operation not permitted`.

Justificación Técnica

Aunque el archivo .ko cuenta con una firma digital estructuralmente válida generada en los pasos previos, la firma fue cifrada utilizando una clave privada cuya contraparte pública (MOK.der) **solo reside en el llavero UEFI de la máquina del autor original**.

Dado que el Secure Boot está activo en el computador del compañero, el kernel tiene una política estricta de validación. Al evaluar el módulo, buscará la clave pública del firmante dentro de su base de datos local de llaves autorizadas (las claves de la distribución precargadas más su base de datos MOK local). Al no encontrar coincidencia con la clave pública del autor original, el subsistema de seguridad determina que el origen del módulo es desconocido y potencialmente peligroso, abortando el proceso para preservar la integridad del sistema.

Solución

Para que el compañero pueda ejecutar el módulo de manera exitosa, el autor original debe compartirle su archivo de clave pública MOK.der. El compañero deberá importar dicha clave pública en su propia máquina ejecutando `sudo mokutil --import MOK.der`, reiniciar su equipo, autorizar la clave en la pantalla azul de la UEFI mediante la introducción de la contraseña asignada, y finalmente proceder con la carga del módulo.

11. Análisis del impacto de las actualizaciones de seguridad en sistemas de arranque dual (Caso de Estudio: Parche SBAT de Microsoft)

A. Consecuencia principal del parche de Microsoft sobre GRUB en sistemas con arranque dual

La consecuencia directa de la aplicación de la actualización de seguridad provista por Microsoft fue el bloqueo absoluto del arranque de las distribuciones de Linux en configuraciones con arranque dual (Dual Boot con Windows). Al intentar seleccionar e iniciar la distribución de Linux desde el menú de GRUB, el mecanismo de control de firmware UEFI interceptaba el proceso y abortaba el inicio, mostrando una advertencia crítica en pantalla: `Verifying shim SBAT data failed: Security Policy Violation`. Este error impedía el acceso a los sistemas operativos Linux cargados en el equipo, dejando las estaciones de trabajo únicamente con la opción de arrancar el entorno Windows.

B. Implicancia de desactivar Secure Boot como contramedida al problema descrito

Si bien desactivar el Arranque Seguro (Secure Boot) desde la configuración de la UEFI mitiga el síntoma de inmediato (permitiendo que el archivo shim y el gestor

GRUB vuelvan a iniciar Linux sin restricciones), esta acción tiene una grave implicancia de seguridad:

Al apagar Secure Boot, se desactiva por completo la cadena de confianza criptográfica del hardware. El sistema de inicio queda vulnerable ante ataques avanzados que utilicen cargadores de arranque maliciosos compilados a medida, modificados o antiguos que contengan fallos graves ya conocidos (como Bootkits o vulnerabilidades históricas de GRUB). Sin Secure Boot activo, cualquier software malicioso que logre acceso de administrador en el sistema operativo puede sobrescribir los sectores de arranque principales e inyectar código persistente que se ejecutará antes de que carguen las defensas habituales del sistema de archivos o el propio antivirus del sistema operativo.

C. Propósito principal del Secure Boot en el proceso de arranque de un sistema

El propósito fundamental de Secure Boot (un estándar desarrollado por la industria tecnológica integrado en la especificación UEFI) es garantizar de manera fehaciente que un dispositivo electrónico inicie su secuencia de encendido utilizando de forma exclusiva software que se mantenga íntegro y que sea de absoluta confianza para el fabricante del hardware (OEM) o el propietario del equipo.

Esto se logra mediante la creación de una cadena de verificación criptográfica basada en llaves asimétricas. Desde el momento en que se presiona el botón de encendido, el firmware de la placa madre valida la firma digital de cada componente de software involucrado en el arranque (la ROM de la tarjeta de video, los drivers internos de la UEFI, el cargador de arranque primario como shim, el cargador de arranque secundario como GRUB, y finalmente la imagen del Kernel de Linux). Si un elemento de la cadena fue alterado, modificado de forma maliciosa por un tercero, carece de firma, o su firma digital coincide con un identificador de clave revocado por fallos de seguridad conocidos (lista de revocación SBAT o dbx), Secure Boot impide su ejecución de forma preventiva para neutralizar la inyección de código malicioso de bajo nivel de forma persistente.